# EAST POINT COLLEGE OF ENGINEERING & TECHNOLOGY

## Department of Computer Science & Engineering

AUTONOMOUS SCHEME AFFILIATED TO VTU | ACCREDITED BY NAAC (A GRADE) & NBA

**COURSE CODE: 24CS51 / BCS501**

# TRUSTLENS AI

## Product-Based Comprehensive Technical Viva-Voce & Project Evaluation Manual

**SUBJECT FACULTY & EVALUATORS**

**Prof. Madhu Shree R**
Assistant Professor, Dept. of CSE

**Dr. Chandramouli H / Dr. Pavan**
Associate Professor / Professor, Dept. of CSE

**STUDENT CANDIDATE & COURSE**

**V Semester B.E. (Computer Science)**
Academic Year: 2026 – 2027

**Product: TrustLens AI**
Evidence-Aware AI Verification Engine

Covers Module 1 (SE Fundamentals & Process Models), Module 2 (Agility & XP Practice), and Module 3 (Project Management & Control).

# TRUSTLENS AI × SEPM (24CS51 / BCS501)

## Comprehensive Product-Based Technical Viva Examination Manual

**Course:** Software Engineering and Project Management (24CS51 / Autonomous Scheme)
**Institution:** East Point College of Engineering & Technology (EPCET), Dept. of Computer Science & Engineering
**Instructors:** Prof. Madhu Shree R / Dr. Chandramouli H / Dr. Pavan
**Candidate:** 5th Semester B.E. (CSE)
**Target Product:** TrustLens AI (Evidence-Aware AI Verification & Fact-Checking Engine)

---

## TABLE OF CONTENTS

1. Actual Product Audit & Reality Check
2. Process Models: Academic Truth & Defense Strategy
3. Module 1: SE Fundamentals & Process Models × TrustLens AI
4. Module 2: Agile Development & Practice Principles × TrustLens AI
5. Module 3: Software Project Management & Control × TrustLens AI
6. Core Technical Architecture × SEPM Concepts
7. Essential Exam Diagrams & Viva Defense Guide
8. 50 Difficult & Challenging Viva Cross-Questions with Model Answers
9. Examiner Traps, Red Lines & How to Answer Honestly

---

## 1. ACTUAL PRODUCT AUDIT & REALITY CHECK

When an external examiner questions your project, they will instantly spot if you claim a theoretical ideal that does not match your codebase. Here is the verified technical autopsy of TrustLens AI:

```
graph TD
    Client["React 18 SPA (Vite :5173)<br/>Chat, Verification Pulse, Attachments, A11y"]
    API["Express 4 REST Backend (:4000)<br/>Auth, History, Calibrator, Router"]
    DB[("SQLite 3 Database<br/>users, conversations, messages, feedback")]
    Gemini["Google Gemini 2.5 Flash API<br/>Search Grounding & JSON Schema"]
    Calibrator["Multi-Factor Confidence Engine<br/>Source, Evidence, Claim, Uncertainty Penalty"]

    Client -->|HTTP / Bearer JWT| API
    API -->|CRUD SQL Queries| DB
    API -->|Prompt + Grounding Tools| Gemini
    Gemini -->|Claims + Grounding Metadata| API
    API -->|Independent Verification Math| Calibrator
    Calibrator -->|Final Calibrated Response| Client
```

## Verified Implementation Breakdown

| Feature Category | Implementation Status | Exact Codebase Location | Academic Defense / Reality |
|---|---|---|---|
| **Claim Extraction & Fact Checking** | **Fully Implemented** | `backend/src/services/trustlens.js`, `gemini.js` | Gemini extracts distinct factual claims, source references, and speculative uncertainties under structured JSON schema. |
| **Confidence Assessment** | **Fully Implemented (Algorithmic)** | `backend/src/lib/confidence.js` | **CRITICAL DEFENSE POINT:** Confidence is **not** an LLM guess. It is computed by a backend heuristic function ( `calibrateTrustAnswer` ) that checks source authority (0.35 weight), evidence sufficiency (0.30 weight), verified claims ratio (0.25 weight), and applies penalties for conflicting data (-25%) and unverified claims. |
| **Multi-turn Chat & History** | **Fully Implemented** | `backend/src/routes/chat.js`, `services/conversations.js` | Full conversation persistence in SQLite ( `conversations` and `messages` tables) with user/guest ownership and soft-delete/export capabilities. |
| **New Chat Isolation** | **Fully Implemented** | `src/App.jsx`, `backend/src/services/conversations.js` | Fresh state on "New Chat" button: resets active conversation ID, empties composer, prevents historical context leakage into new prompt. |
| **Authentication & Sessions** | **Fully Implemented** | `backend/src/routes/auth.js`, `services/users.js` | Scrypt password hashing, HMAC-SHA256 JWT tokens, Guest sessions with zero-loss upgrade, and Google OAuth 2.0 Identity Services. |
| **File Attachment Processing** | **Fully Implemented** | `src/components/TrustComposer.jsx`, `backend/src/routes/chat.js` | Multi-file client uploader (PDF, DOCX, TXT, PNG, JPG). Base64 encoded and ingested into Gemini multimodal API. |
| **Voice Input** | **Fully Implemented** | `src/components/TrustComposer.jsx` | Browser-native Web Speech API ( `SpeechRecognition` / `webkitSpeechRecognition` ) with interim transcript |

| Feature Category | Implementation Status | Exact Codebase Location | Academic Defense / Reality |
|---|---|---|---|
| | | | streaming and microphone permission handling. |
| **Accessibility (a11y)** | **Fully Implemented** | `src/components/AccessibilityModal.jsx`, `src/styles.css` | WCAG AAA high contrast mode, OpenDyslexic font toggle, reduced motion preference, scalable font sizes, full keyboard navigation. |
| **Verification Pulse Loader** | **Fully Implemented** | `src/components/VerificationPulseLoader.jsx` | Multi-stage verification status indicator (parsing, grounding search, claims evaluation, confidence calibration). |
| **Automated Live Grounding** | **Fully Implemented** | `backend/src/services/gemini.js` | Uses Google Search Dynamic Retrieval tool in Gemini API to fetch live real-world citations. |
| **Automated Regression Test Suite** | **Fully Implemented** | `scripts/test-*.js`, `scripts/autopsy-test-suite.js` | 89+ automated tests across API endpoints, edge cases, SQL injections, and Puppeteer visual regressions. |
| **Distributed Microservices / Multi-Node DB** | **Planned / Future Scope** | Single SQLite node, monolithic backend | Single-instance SQLite DB. For enterprise scale (10,000+ concurrent users), PostgreSQL with read-replicas and Redis caching are required. |

---

## 2. PROCESS MODELS: ACADEMIC TRUTH & DEFENSE STRATEGY

### How to Answer: "Which SDLC Model Did You Follow?"

> [!CAUTION] **DO NOT CLAIM:** "*We followed pure Waterfall*" (an AI app with changing UI cannot be Waterfall), NOR "*We did textbook Scrum with a Scrum Master, daily 15-minute standups, and 30-day timeboxed sprints*" (if you were working individually or in an academic pair, claiming you had a designated Scrum Master is easily dismantled by examiners).

### The Best Academic Answer:

> *"Madam, for TrustLens AI, we followed an **Agile Evolutionary-Incremental Model**, drawing principles from **Extreme Programming (XP)** and **Agile Prototyping**.
>
> Because TrustLens integrates a non-deterministic generative AI API (Google Gemini) with strict deterministic verification requirements, our system had high requirement volatility and technical unknowns. We could not specify all prompt constraints, verification weights, and UI components up-front in a traditional Waterfall manner.
>
> Therefore, we structured development into **four distinct increments**:
>
> 1. **Increment 1 (Core LLM & API Spike):** Direct Node.js to Gemini integration with basic prompt-response mechanism.
> 2. **Increment 2 (Evidence & Confidence Calibration Engine):** Added claim extraction, Google Search grounding, and deterministic mathematical confidence scoring ( `calibrateTrustAnswer` ).
> 3. **Increment 3 (Interactive Workspace & Accessibility UI):** Built the React SPA, TrustComposer, Verification Pulse loader, and WCAG accessibility modes.
> 4. **Increment 4 (Security, Persistence & Production Hardening):** Added SQLite persistence, JWT authentication, Google OAuth 2.0, guest session upgrade, and automated regression testing.
>
> This approach allowed us to flatten the **Agile Cost-of-Change curve** and test high-risk AI failure modes early through rapid spike solutions."*

---

## 3. MODULE 1: SE FUNDAMENTALS & PROCESS MODELS × TRUSTLENS AI

### 1. Characteristics of Software (Software vs Hardware)

- **Theory Question:** What are the fundamental characteristics of software that distinguish it from hardware?
- **Product-Based Question:** In what ways did TrustLens AI behave as an engineered product rather than a manufactured artifact during development?
- **Justification Question:** Your notes state: "*Software doesn't wear out; it deteriorates due to change.*" How does this apply to TrustLens AI?
- **Implementation Question:** When Google updated the Gemini SDK or when frontend requirements changed from dark mode to light mode, where did deterioration risk occur in TrustLens AI?
- **Challenge Question:** Why can't you simply buy "spare parts" for a broken verification algorithm in TrustLens AI?
- **Follow-up Question:** How does the bathtub curve of hardware failure differ from the idealized and actual failure curves of TrustLens AI?

### Model Viva Answer

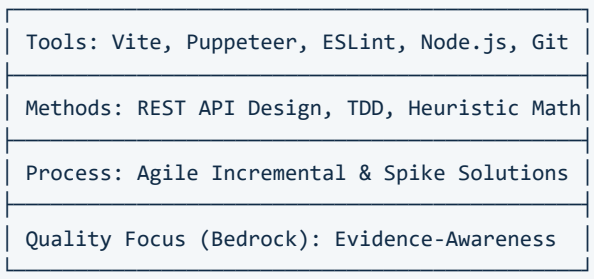
> **Short Direct Answer:** Software is engineered, custom-built, and deteriorates over time due to environmental and requirement changes rather than physical wear and tear.
> **Detailed Viva Answer:** Unlike hardware, where manufacturing introduces defects and physical components wear out over time following the bathtub curve, TrustLens AI was custom-engineered. Its costs were concentrated in design, prompt engineering, and algorithmic calibration rather than mass production.
> TrustLens AI does not physically degrade, but it suffers from **software deterioration**: as the external Gemini API evolves, browser standards shift, or new edge-case queries arise, code defects and architectural misalignments emerge. For instance, when we updated token authentication or modified the theme palette, changes in one CSS class or database column could introduce regression errors unless mitigated through maintenance and refactoring.
> **TrustLens Example:** In `backend/src/db.js`, when we added Google OAuth, SQLite did not support easy table dropping without losing data. We had to write idempotent schema migrations (`ALTER TABLE users ADD COLUMN google_id TEXT`) to adapt the system without deteriorating existing user records.

---

## 2. Software Engineering Layered Technology

- **Theory Question:** Explain "Software Engineering – A Layered Technology" and its four constituent layers.
- **Product-Based Question:** Map the four layers of software engineering (Quality Focus, Process, Methods, Tools) to your TrustLens AI project.
- **Justification Question:** Why is Quality Focus considered the bedrock layer for a product like TrustLens AI?
- **Implementation Question:** Which CASE tools and developer toolsets did you utilize to build and verify TrustLens AI?

### Model Viva Answer




> **Explanation for Viva:**
>
> 1. **Quality Focus (Bedrock):** In TrustLens AI, the core commitment is answer reliability and verification truthfulness. If the AI hallucinates confidence, the entire product value collapses.
> 2. **Process:** We adopted an Agile Incremental process flow, delivering core chat first, then claim calibration, then authentication, and finally OAuth and accessibility.
> 3. **Methods:** We used structured RESTful API design, schema validation, multi-factor heuristic scoring in `backend/src/lib/confidence.js`, and component-based UI engineering with React.
> 4. **Tools (CASE):** We used modern Computer-Aided Software Engineering tools: Vite for rapid HMR bundling, SQLite3 for embedded relational persistence, Puppeteer for automated visual regression testing, and Postman/Node test scripts for automated endpoint verification.

---

## 3. Generic Process Framework Activities (C-P-M-C-D)

- **Theory Question:** What are the five generic framework activities in a software process?
- **Product-Based Question:** How were Communication, Planning, Modelling, Construction, and Deployment executed during TrustLens AI development?
- **Implementation Question:** What models were created during the Modelling activity of TrustLens AI?

### Model Viva Answer

> 1. **Communication:** Identifying the user need for an evidence-grounded AI assistant rather than a black-box chatbot that invents facts. Gathering requirements on accessibility (WCAG) and file processing.
> 2. **Planning:** Defining the task set: API integration, database schema, confidence algorithm, and UI layout. Assessing technical risks (Gemini API rate limits, network latency).
> 3. **Modelling:** Constructing Entity-Relationship models for SQLite ( `users` , `conversations` , `messages` ), Data Flow diagrams for claim verification, and UI state models for the chat composer.
> 4. **Construction:** Coding the frontend in React 18, backend in Express, and writing 89+ unit and integration tests (such as `scripts/test-google-auth.js` ).
> 5. **Deployment:** Bundling the frontend via Vite ( `npm run build` ), deploying the Node backend server on port 4000, and validating live responses.

---

## 4. Umbrella Activities in TrustLens AI

- **Theory Question:** What are umbrella activities? List any five.
- **Product-Based Question:** How did you conduct Software Quality Assurance (SQA) and Software Configuration Management (SCM) in TrustLens AI?

### Model Viva Answer

> **Theory:** Umbrella activities are applied throughout the software life cycle to manage progress, quality, change, and risk, independent of any single framework activity.
> **Application to TrustLens AI:**
>
> 1. **Software Configuration Management (SCM):** Git version control tracking every branch, commit, and artifact version; `.env` configuration separation.
> 2. **Software Quality Assurance (SQA):** Automated endpoint regression test suites verifying HTTP response codes, JSON schema conformity, and token expiration.
> 3. **Technical Reviews:** Performing code walk-throughs to inspect prompt injection vulnerabilities, rate-limiting, and error catch-blocks in `backend/src/routes/chat.js` .
> 4. **Risk Management:** Proactively implementing timeout wrappers ( `withTimeout` at 35s in `gemini.js` ) and fallback heuristic scoring when Google Search grounding produces no hits.
> 5. **Measurement & Metrics:** Tracking API response latency, SQLite query duration, and test pass/fail rates.

---

## 5. David Hooker's Seven Principles

- **Theory Question:** State Hooker's seven principles of software engineering practice.
- **Product-Based Question:** Give a specific example where you applied Hooker's Principle 2: *KISS (Keep It Simple, Stupid!)* in TrustLens AI.
- **Implementation Question:** How does TrustLens AI implement Principle 6: *Plan Ahead for Reuse*?

### Model Viva Answer

> **KISS Principle in TrustLens:** Instead of configuring an over-engineered microservices cluster with Kafka and Redis for an academic prototype, we chose a lightweight, highly efficient SQLite database with synchronous write-ahead logging (WAL) and an Express modular router. This reduced complexity while fulfilling all functional requirements.
> **Plan Ahead for Reuse:** The confidence calibration engine `calibrateTrustAnswer()` is written as a pure, decoupled JavaScript utility with zero external dependencies. It can be reused in any Node.js service, CLI tool, or frontend application without modification.

---

## 6. Software Myths & Realities

- **Theory Question:** Explain one Management myth, one Customer myth, and one Practitioner myth along with their realities.
- **Product-Based Question:** Which software myth did you experience firsthand while developing TrustLens AI?

### Model Viva Answer

> **Practitioner Myth Experienced:** "*Once we write the program and get the chat working, our job is done.*"
> **Reality in TrustLens:** Building the basic Gemini chat endpoint took only 15% of our time. 85% of the total engineering effort was spent on input validation, token session management, edge-case handling (empty inputs, network timeouts, corrupt file uploads), light/dark mode contrast tuning, and writing automated tests.
> **Customer Myth:** "*A general statement like 'Make an AI fact checker' is sufficient to begin writing code.*"
> **Reality in TrustLens:** Without formalizing exact claim schema, confidence formulas, and source trust scoring, the AI returned vague, unverifiable answers. Detailed requirement elicitation was indispensable.

---

## 7. Process Models Comparison (Waterfall vs V-Model vs Incremental vs Spiral)

```
graph LR
    subgraph Waterfall
    A[Req] --> B[Design] --> C[Code] --> D[Test]
    end
    subgraph Incremental
    I1[Increment 1: Core Chat] --> I2[Increment 2: Evidence Scoring] --> I3[Increment 3: Auth & UI]
    end
    subgraph Spiral
    S1[Objective] --> S2[Risk Analysis] --> S3[Engineering] --> S4[Evaluation]
    end
```

| Dimension | Waterfall Model | Incremental Model (Used in TrustLens) | Spiral Model (Boehm) |
|---|---|---|---|
| **Requirement Stability** | Must be 100% frozen upfront | Evolves with each delivered increment | Evolves through risk-driven spiral circuits |
| **Working Software** | Late in the life cycle | Early working increments delivered continuously | Working prototypes at milestone revolutions |
| **Risk Handling** | Poor; late discovery of defects | Moderate; risks partitioned into increments | Exceptional; explicit risk analysis quadrant |
| **Suitability for TrustLens** | **Unsuitable:** Gemini prompts and AI output formats could not be frozen upfront. | **Best Fit:** Allowed us to ship working chat first, then increment with grounding, UI, and auth. | **Viable Alternative:** Excellent for large-scale enterprise AI due to API cost/safety risks, but overkill for our team size. |

---

# 4. MODULE 2: AGILE DEVELOPMENT & PRACTICE PRINCIPLES × TRUSTLENS AI

## 1. Agility & The Cost-of-Change Curve

- **Theory Question:** What is agility in software engineering? How does an agile process flatten the cost-of-change curve?
- **Product-Based Question:** Draw and explain the cost-of-change curve for TrustLens AI when transitioning from a dark-themed demo to a full light/dark accessible design system.
- **Justification Question:** If a major requirement change occurred in week 4 of TrustLens AI, how did modularity prevent exponential cost growth?

### Model Viva Answer

> **Theory:** In conventional development, the cost of making a change grows non-linearly (exponentially) as the project progresses because architectural frameworks, database schemas, and documentation are already frozen. Agile flattens this curve through short iterations (1–4 weeks), continuous unit testing, refactoring, and frequent integration.
> **TrustLens Example:** When we decided to add Google OAuth and guest session upgrades late in the project, our decoupled architecture ( `backend/src/routes/auth.js` separated from `backend/src/services/users.js` ) meant we only added an idempotent database column and an auth endpoint. We did not need to rewrite the chat engine or redesign the database.

---

## 2. Extreme Programming (XP) in TrustLens AI

- **Theory Question:** What are the five core values and four framework activities of Extreme Programming (XP)?
- **Product-Based Question:** Explain how **Spike Solutions** and **Refactoring** were used in TrustLens AI.

- **Implementation Question:** What is Pair Programming, and where would it be most beneficial in TrustLens AI?

### Model Viva Answer

> **The 5 XP Values:** Communication, Simplicity, Feedback, Courage, Respect.
> **The 4 XP Activities:** Planning, Design, Coding, Testing.
> **Spike Solution in TrustLens:** In XP, when a team encounters a difficult, unknown design problem, they build an immediate, disposable operational prototype called a **Spike Solution**. In TrustLens AI, before integrating Gemini into our React UI, we built a standalone 30-line Node.js spike script ( `scripts/test-gemini-grounding.js` ) to test whether the `@google/genai` SDK properly returned Google Search grounding chunks in JSON format. Only once the spike confirmed feasibility did we implement the production service.
> **Refactoring in TrustLens:** We refactored `src/App.jsx` by extracting the composer logic into `src/components/TrustComposer.jsx` and the verification animation into `src/components/VerificationPulseLoader.jsx` . This improved internal code cohesion and maintainability without altering external user behavior.

---

## 3. Scrum Framework Applied to TrustLens AI

- **Theory Question:** Explain Scrum framework activities, roles, and ceremonies (Product Backlog, Sprint Backlog, Sprint, Daily Scrum, Sprint Review/Demo).
- **Product-Based Question:** If you were to organize TrustLens AI under Scrum, what would be the Product Backlog, and what would constitute a 2-week Sprint Backlog?
- **Examiner Question:** What are the three questions answered in every 15-minute Daily Scrum meeting?

### Model Viva Answer

> **The 3 Daily Scrum Questions:**
>
> 1. *What did you do since the last team meeting?* (e.g., implemented the Web Speech API in TrustComposer).
> 2. *What obstacles are you encountering?* (e.g., microphone permission prompt blocked on insecure HTTP localhost).
> 3. *What do you plan to accomplish by the next team meeting?* (e.g., write automated unit test for speech silence timeout).
>
> **Product Backlog for TrustLens AI:**
>
> - User Story 1: As a researcher, I want claims extracted from AI answers so that I can independently verify each statement.
> - User Story 2: As an applicant, I want source authority trust scores so that I know if information comes from an official gazette or an unverified blog.
> - User Story 3: As a visually impaired student, I want a high-contrast mode with OpenDyslexic font support to read answers comfortably.
>
> **Sprint Backlog (Sprint 2 - 14 Days):**
>
> - Implement `calibrateTrustAnswer` heuristic in `confidence.js` .
> - Add search grounding citation pills in chat message UI.
> - Write regression test verifying 80% confidence cap when sources conflict.

---

### 4. Feature Driven Development (FDD) & Peter Coad's Template

- **Theory Question:** What is Feature Driven Development (FDD)? What is Peter Coad's template for defining a feature?
- **Product-Based Question:** Express two key capabilities of TrustLens AI using Peter Coad's feature template.

**Model Viva Answer**

> **Peter Coad's Template:** `<action> the <result> <by|for|of|to> a(n) <object>`
> **TrustLens Feature 1:** `Extract` the `factual claims` `from` a `user query response`.
> **TrustLens Feature 2:** `Calibrate` the `confidence score` `for` an `evidence-backed answer`.
> **TrustLens Feature 3:** `Upgrade` the `guest session` `to` a `registered user account`.

---

### 5. Construction & Testing Principles (Glen Myers & Davis)

- **Theory Question:** What are Glen Myers' testing rules, and what are Davis's core testing principles?
- **Product-Based Question:** How does Davis's principle "*Testing should begin in the small and progress toward testing in the large*" apply to TrustLens AI?

**Model Viva Answer**

> **Glen Myers' Testing Rule:** "*Testing is a process of executing a program with the intent of finding an error. A good test case is one that has a high probability of finding an as-yet undiscovered error.*"
> **Testing "In the Small" to "In the Large" in TrustLens AI:**
>
> 1. **In the Small (Unit Testing):** Testing the pure math functions in `backend/src/lib/confidence.js` with mock sources to verify that conflicting evidence deducts exactly 25% confidence.
> 2. **Sub-System Integration Testing:** Testing `backend/src/routes/auth.js` with supertest/fetch to verify that invalid passwords return HTTP 401 and valid credentials return a signed JWT.
> 3. **In the Large (System & UI Testing):** Using Puppeteer in `scripts/test-google-auth.js` to open a real browser, load `http://localhost:5173`, simulate clicking "Continue with Google", verify DOM rendering, and assert that the user profile updates.

---

## 5. MODULE 3: SOFTWARE PROJECT MANAGEMENT & CONTROL × TRUSTLENS AI

### 1. The Four P's of Management (People, Product, Process, Project)

- **Theory Question:** Explain the Four P's of software project management.
- **Product-Based Question:** Detail the Four P's as specifically applied to TrustLens AI.

### Model Viva Answer

1. **People:** End users (students, applicants, researchers verifying factual claims), Developers (full-stack engineering, prompt design), and Evaluators/Stakeholders (SEPM faculty).
2. **Product:** TrustLens AI — an evidence-grounded verification platform consisting of a React SPA frontend, an Express Node.js backend, SQLite database, and Google Gemini API integration.
3. **Process:** An Agile Evolutionary-Incremental process framework encompassing iterative user-story implementation, spike solutions for AI grounding, and continuous automated testing.
4. **Project:** A 12-week development effort adhering to resource constraints (single developer/pair team, free-tier API quotas, zero-budget open-source stack) tracked against milestone deadlines.

---

## 2. Five Variables of Project Control & The Trade-off Triangle

- **Theory Question:** What are the five variables of project control? Explain the Trade-off Triangle.
- **Product-Based Question:** If your project submission deadline was reduced by 50% (Time variable compressed), how would you adjust the Trade-off Triangle for TrustLens AI?

```
graph TD
    Cost["COST<br/>(Resources / Team)"] --- Time["TIME<br/>(Schedule / Deadline)"]
    Time --- Scope["SCOPE<br/>(Features / Requirements)"]
    Scope --- Cost
    subgraph Center
    Quality["QUALITY<br/>(Fit-to-Purpose / Reliability)"]
    end
```

### Model Viva Answer

**The 5 Variables:** Time, Cost, Quality, Scope, Risk.
**Trade-Off Triangle Defense:** In the Trade-off Triangle, Time, Cost, and Scope represent the three primary forces, with Quality at the center. If one factor is fixed, the others must vary.
*"Madam, if our deadline were cut by 50%, we would **never compromise Quality**, because in an AI verification engine, delivering poor quality means delivering misleading or hallucinated facts, which destroys product viability. Because our team size and budget (Cost) are fixed, we would **negotiate Scope** using the **DSDM 80/20 Pareto principle**. We would preserve core verification, claim extraction, and chat (the 80% customer value), while deferring secondary scope like multi-format file attachment parsing or export to CSV to a subsequent release."*

---

## 3. Barry Boehm's W5HH Principle

- **Theory Question:** What is Barry Boehm's W5HH principle?
- **Product-Based Question:** Answer all seven questions of the W5HH principle for TrustLens AI.

### Model Viva Answer

| Question | SEPM Concept | TrustLens AI Implementation |
|---|---|---|
| **Why is the system being built?** | Business Case / Purpose | To eliminate generative AI hallucinations and provide evidence-calibrated answers before users make consequential decisions. |
| **What will be done?** | Scope & Deliverables | A full-stack web application with claim extraction, source grounding, multi-factor confidence scoring, and chat persistence. |
| **When will it be done?** | Project Schedule | Delivered across 4 development increments over a 12-week academic semester. |
| **Who is responsible?** | Staffing & Stakeholders | 5th Semester CSE candidate (Architecture, Frontend, Backend, Calibration Engine). |
| **Where are they organizationally located?** | Resource Allocation | East Point College of Engineering & Technology, Dept. of CSE. |
| **How will the job be done technically & managerially?** | Process & Architecture | React + Node.js + SQLite architecture executed via Agile Incremental methodology with automated testing. |
| **How much of each resource is needed?** | Cost & Estimation | 1 developer workstation, Google Gemini API developer key, open-source npm dependencies, 0 capital expenditure. |

---

## 4. Software Project Categorization (Module 3 / Notes §1.7)

- **Theory Question:** What are the different ways of categorizing software projects according to your notes?
- **Product-Based Question:** Categorize TrustLens AI across all dimensions from your syllabus.

### Model Viva Answer

> 1. **Software Product vs. Software Service:** TrustLens AI is a **Software Product Development project (Generic)** designed for a broad consumer audience rather than a tailored client service.
> 2. **Voluntary vs. Compulsory:** It is a **Voluntary System** — users choose to consult TrustLens to verify information; therefore, UX, aesthetic appeal, and immediate credibility are crucial to retain users.
> 3. **Information System vs. Embedded:** It is an **Information System** operating over the web, managing text, documents, and database records, not controlling physical hardware.
> 4. **In-house vs. Outsourced:** Developed **In-House** with third-party cloud API services (Gemini).

---

## 5. Stakeholder Theory & Boehm & Ross "Theory W"

- **Theory Question:** What is Boehm and Ross's "Theory W" in project management?
- **Product-Based Question:** Identify the internal and external stakeholders of TrustLens AI and show how a "Win-Win" state is achieved.

**Model Viva Answer**

> **Theory W (Win-Win):** A project management principle where the manager creates conditions where all key parties benefit from the project, giving everyone an interest in its success.
> **TrustLens Stakeholder Mapping:**
>
> - **End-Users (Students & Researchers):** *Win:* Receive transparent, evidence-checked answers with clear citations, avoiding costly application mistakes.
> - **Developers:** *Win:* Built a demonstrable, production-grade full-stack portfolio product incorporating modern AI grounding and full test coverage.
> - **Academic Evaluators (Faculty):** *Win:* Assesses a real-world, working implementation that rigorously adheres to syllabus concepts (24CS51).

---

## 6. CORE TECHNICAL ARCHITECTURE × SEPM CONCEPTS

### 1. Request-Response Verification Pipeline (Data Flow)

When the examiner asks: "*What happens under the hood when a user asks a question in TrustLens AI?*"

```
sequenceDiagram
    autonumber
    actor User as User
    participant Browser as React SPA (Vite)
    participant Server as Express Server (:4000)
    participant Gemini as Google Gemini 2.5 Flash
    participant Calibrator as Confidence Calibrator
    participant DB as SQLite DB

    User->>Browser: Enters query & clicks Send
    Browser->>Browser: Render "Verification Pulse" stage 1
    Browser->>Server: POST /api/chat/message { conversationId, content }
    Server->>DB: INSERT into messages (role='user')
    Server->>Gemini: callGemini({ prompt, enableSearch: true, schema })
    Gemini->>Gemini: Google Search Dynamic Retrieval
    Gemini-->>Server: JSON { answer, claims, sources, uncertainties }
    Server->>Calibrator: calibrateTrustAnswer(claims, sources, uncertainties)
    Note over Calibrator: Source Score (0.35) + Evidence (0.30) + Claims (0.25) - Conflict Penalty
    Calibrator-->>Server: { pct: 82, level: 'High', breakdown }
    Server->>DB: INSERT into messages (role='assistant', confidence, sources)
    Server-->>Browser: HTTP 200 { message, confidence, claims, sources }
    Browser->>Browser: Replace pulse loader with active answer & citation pills
```

### 2. Deterministic Verification vs. Probabilistic LLMs

- **Examiner Cross-Question:** "*If Gemini generates the answer, didn't Gemini also generate the confidence score? How can you trust an AI that rates its own accuracy?*"
- **The Winning Defense:**

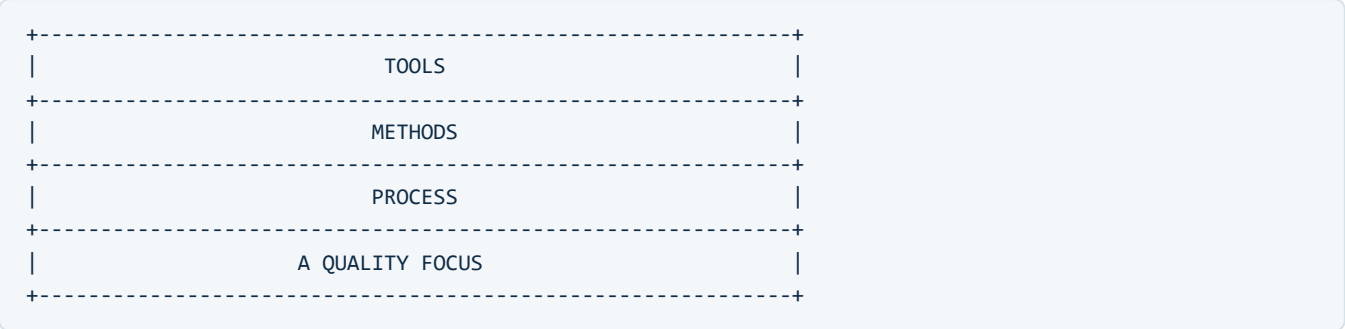
> *"Madam, that is the central design problem we solved in TrustLens AI. We **do not allow the LLM to self-report its confidence**.
> In `backend/src/lib/confidence.js`, we built an independent, deterministic **Confidence Calibration Engine**.
> The LLM is only used as a parser to extract factual claims and retrieve search grounding chunks. Our backend code then runs a mathematical formula:
>
> $$\text{Score} = (0.35 \times \text{SourceScore}) + (0.30 \times \text{EvidenceScore}) + (0.25 \times \text{ClaimScore}) - \text{Penalty}$$
>
> If sources conflict, our code forcibly deducts 25 points. If there are zero official sources, the score is capped. The LLM cannot override this rule. Thus, verification is deterministic and auditable."*

---

## 7. ESSENTIAL EXAM DIAGRAMS & VIVA DEFENSE GUIDE

You may be asked to draw any of the following diagrams on a whiteboard or sheet of paper.

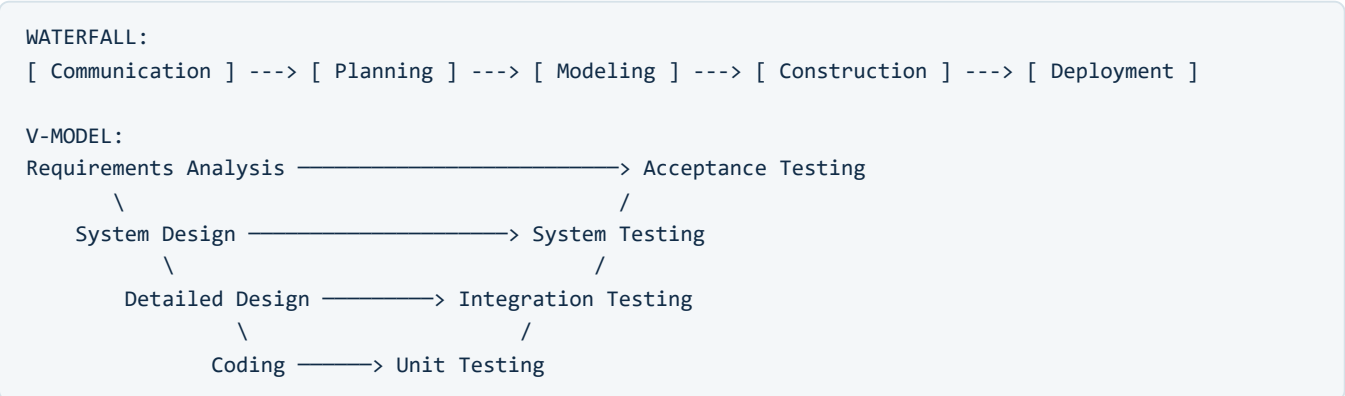
### Diagram 1: The Generic Process Framework & Layered Technology




- **Viva Question:** Why is Quality placed at the bottom?
- **Answer:** It serves as the bedrock foundation. Without quality focus, any process, method, or tool will still result in software failure.

---

### Diagram 2: The Waterfall Model vs. The V-Model




- **Viva Question:** How does the V-Model improve upon the classic Waterfall?
- **Answer:** It explicitly visualizes the relationship between quality assurance actions (testing) and early design/analysis actions, enforcing that test planning occurs long before coding commences (Davis Principle

2).

---

### Diagram 3: Barry Boehm's Spiral Model

```
                        Determine objectives,
                          alternatives, constraints
                                    |
  Quadrant 1                        |              Quadrant 2
                                    |   Identify & resolve risks
                                    |   (Risk Analysis & Prototyping)
-------------------------+----------+-------------------------
  Quadrant 4                        |              Quadrant 3
                                    |   Develop & verify next-level
  Plan next phases                  |   product (Engineering)
                                    |
```

- **Viva Question:** What are the two distinguishing features of the Spiral Model?
- **Answer:** (1) A cyclic approach for incrementally growing a system's definition while decreasing risk. (2) A set of anchor-point milestones ensuring stakeholder commitment.

---

### Diagram 4: Extreme Programming (XP) Process Flow

```
[ User Stories ] ──> [ Release Plan ] ──> [ Spike Solutions ] ──> [ Simple Design ]
                                                                          |
[ Acceptance Tests ] <── [ Software Increment ] <── [ Continuous Test ] <── [ Pair Coding (TDD) ]
```

- **Viva Question:** When does testing begin in XP?
- **Answer:** Before coding commences. XP enforces Test-Driven Development (TDD) where automated unit tests are written first to specify the behavior of each class or user story.

---

### Diagram 5: Scrum Process Flow

```
[ Product Backlog ] ──> [ Sprint Backlog ] ──> [ 30-Day Sprint (Daily 15-min Scrum) ] ──> [ Working Increme
```

---

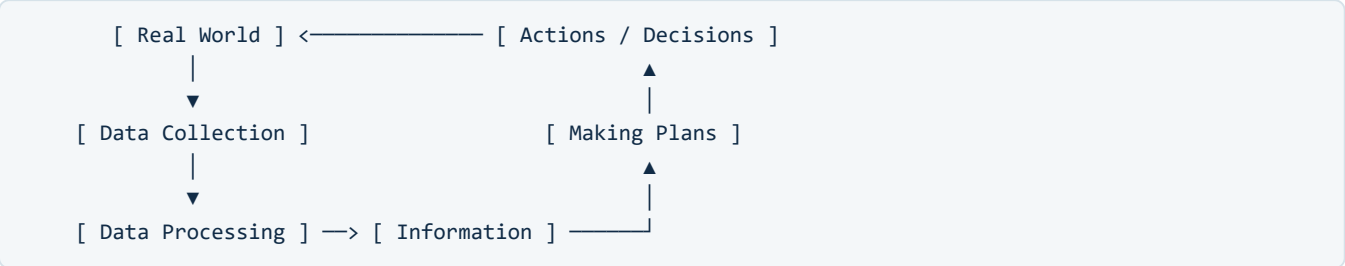
### Diagram 6: Project Management Life Cycle vs. SDLC (Notes Fig 1.7 & 1.8)

```
PMLC:   [ Initiation ] ──> [ Planning ] ──> [ Execution & Control ] ──> [ Closure ]
              |                  |                        |                    |
SDLC:         |          [Requirements] ──> [Design] ──> [Coding/Testing] ──> [Deployment]
```

- **Viva Question:** Does PMLC end when coding is done?
- **Answer:** No. PMLC starts before SDLC (during feasibility study and project chartering) and continues through project closure, vendor payment settlements, and post-implementation reviews.

---

### Diagram 7: The Project Control Cycle (Notes Fig 1.4 / Page 20)




---

## 8. 50 DIFFICULT & CHALLENGING VIVA CROSS-QUESTIONS WITH MODEL ANSWERS

### Category A: Core Product & AI Verification Truth (Q1–Q12)

**Q1: What makes TrustLens AI different from ChatGPT or Perplexity?**

- **Short Answer:** ChatGPT generates authoritative-sounding text without guaranteed citations; TrustLens decomposes every answer into distinct claims and evaluates each against verified evidence.
- **Detailed Answer:** Standard LLMs operate on next-token prediction, which leads to hallucinations and unsupported assertions. TrustLens AI acts as a transparent verification layer. It receives the query, runs Google Search grounding to retrieve current factual records, parses the output into atomic claims, calculates an explainable confidence percentage using a deterministic formula, and visually presents supporting vs. conflicting evidence.
- **SEPM Concept:** Product vs. Vehicle; Verification and Validation (ISO 12207).

**Q2: What happens if two primary sources give conflicting information?**

- **Short Answer:** TrustLens detects the contradiction, deducts a 25% penalty from the confidence score, flags the conflict in amber/rose, and advises the user to consult primary gazettes.
- **Detailed Answer:** In `backend/src/lib/confidence.js`, the `calibrateTrustAnswer` function accepts a `conflicts` array. If sources disagree, `conflicting: true` is triggered, immediately subtracting 25% from the final confidence percentage and capping the status at Medium or Low.
- **SEPM Concept:** Risk Mitigation; Hooker's Principle 1 (Provide value to the user).

**Q3: What if the Gemini API key is exhausted or Google servers are down?**

- **Short Answer:** The system fails gracefully with an HTTP 503 error, a friendly user alert, and maintains full offline access to past chat history.
- **Detailed Answer:** In `backend/src/services/gemini.js`, we wrapped all API invocations with `withTimeout(35000)`. If the request times out or returns a quota exhaustion error, the backend catches it and returns an explanatory HTTP error payload rather than crashing the Node server.
- **SEPM Concept:** Software Reliability; Exception Handling; Umbrella Activity (Risk Management).

**Q4: How do you prevent old conversation context from corrupting a "New Chat"?**

- **Short Answer:** When "New Chat" is clicked, all active conversation IDs, in-memory message states, and attachment trays are reset in React state.

- **Detailed Answer:** In `src/App.jsx`, the `startNewConversation()` function sets `activeId` to `null` and initializes an empty messages array. In the backend, each message is strictly scoped by `conversation_id`. Queries to Gemini only include history corresponding to that exact ID.
- **SEPM Concept:** Information Domain Partitioning; Coupling and Cohesion.

**Q5: Can a user trick TrustLens into verifying a completely fabricated claim?**

- **Short Answer:** No; if a claim has no verifiable sources or search grounding evidence, the confidence score automatically plummets to 15% (Low).
- **Detailed Answer:** If a user submits a fake claim, the search grounding tool finds no supporting authority. Our calibrator assigns a base score of 15% for zero sources. Even if the LLM produces text, the UI flags it with a red warning badge indicating "Insufficient Supporting Evidence".
- **SEPM Concept:** SQA Quality Filter; Validation Testing.

**Q6: How is user voice input processed? Is audio sent to Gemini?**

- **Short Answer:** No audio files are sent over the network. Voice input uses the browser's native Web Speech API to perform speech-to-text on the client side.
- **Detailed Answer:** We implemented `webkitSpeechRecognition` in `src/components/TrustComposer.jsx`. It streams interim transcripts into the text area in real-time. Only the final transcribed text is sent to the backend. This minimizes bandwidth and protects audio privacy.
- **SEPM Concept:** Architectural Modularity; Resource Optimization.

**Q7: What file formats does TrustLens support for verification, and how are they parsed?**

- **Short Answer:** It supports PDF, DOCX, TXT, PNG, JPG, and JPEG via client-side file reading and Gemini multimodal ingestion.
- **Detailed Answer:** In `src/components/TrustComposer.jsx`, files are converted to Base64 data buffers. For images and documents, Gemini's multimodal API directly analyzes the textual and tabular content, matching statements against public search records.
- **SEPM Concept:** Information Transfer; Interface Design (Principle 4).

**Q8: What database are you using, and why didn't you use MongoDB?**

- **Short Answer:** We chose SQLite 3 for ACID compliance, zero-latency local relational queries, and zero-configuration embedded deployment.
- **Detailed Answer:** TrustLens has clear relational structures: Users have many Conversations, and Conversations have many Messages. SQLite with Write-Ahead Logging (WAL) provides strict foreign key integrity, fast indexing, and zero administrative overhead, whereas MongoDB would add unnecessary connection latency and unneeded schema-less overhead.
- **SEPM Concept:** Data Design (Design Modeling Principle 3); KISS Principle.

**Q9: How are passwords stored and authenticated?**

- **Short Answer:** Passwords are never stored in plaintext; they are hashed using Node.js's crypto `scrypt` algorithm with unique cryptographic salts.
- **Detailed Answer:** In `backend/src/services/users.js`, we use `crypto.scryptSync` with random salt generation. User sessions are verified using HMAC-SHA256 JWT tokens with a 30-day expiration sent via `Authorization: Bearer` headers.

- **SEPM Concept:** Security as a Non-Functional Requirement; Defensive Engineering.

**Q10: How does Google OAuth work alongside traditional password authentication?**

- **Short Answer:** It uses Google Identity Services to verify Google ID tokens on the backend, automatically linking accounts by email without creating duplicates.
- **Detailed Answer:** When a user clicks "Continue with Google", Google returns an ID token credential. Our backend `/api/auth/google` verifies the token with Google's OAuth endpoints. If the email matches an existing user, it links `google_id`; if they were a guest, it upgrades their row; if new, it creates a profile.
- **SEPM Concept:** Component Reusability; External Interface Integration.

**Q11: How do you support users with visual or reading impairments?**

- **Short Answer:** Through a dedicated Accessibility Modal providing WCAG AAA High Contrast mode, OpenDyslexic font, and reduced animation modes.
- **Detailed Answer:** In `src/components/AccessibilityModal.jsx`, we persist accessibility preferences in `localStorage` and apply CSS root variables for enhanced contrast ratios, dyslexia-friendly letter spacing, and CSS media queries for `prefers-reduced-motion`.
- **SEPM Concept:** Human-Computer Interaction (HCI); User-Centered Design.

**Q12: What is the biggest technical limitation of TrustLens AI today?**

- **Short Answer:** Reliance on external search engine indexing latency, meaning events that happened 5 minutes ago may lack immediate official citations.
- **Detailed Answer:** Because TrustLens uses Google Search dynamic grounding, if an institution publishes a circular or rule change that has not yet been indexed by search crawlers, our calibrator flags the evidence as insufficient or unverified.
- **SEPM Concept:** Project Constraints; Environmental Dependencies.

---

## Category B: SEPM Process Models & Methodologies (Q13–Q24)

**Q13: Why is the Waterfall model completely unsuitable for TrustLens AI?**

- **Short Answer:** Waterfall demands 100% frozen requirements upfront, whereas generative AI prompt engineering and UI interactions require continuous iterative refinement.
- **Detailed Answer:** In Waterfall, customer requirements must be completely specified before design begins, and working software appears only at the very end. With TrustLens AI, how Gemini formatted grounding metadata and how users interacted with claim cards were discovered empirically through testing, making a linear, non-iterative model a recipe for project failure.
- **SEPM Concept:** Waterfall Model Limitations; Natural Uncertainty in Software.

**Q14: Could you have used the Prototyping model exclusively?**

- **Short Answer:** Prototyping was used for the UI and AI prompts, but using it exclusively risks sacrificing database maintainability and architectural rigor.
- **Detailed Answer:** As noted in Module 1, customers may mistake a prototype for a finished product, unaware that underlying code may have shortcuts. We used prototyping to validate user interaction and

prompt schemas, but transitioned to an engineered incremental architecture for production persistence and security.

- **SEPM Concept:** Prototyping Model; Evolutionary Engineering.

**Q15: How does Barry Boehm's Spiral Model apply to TrustLens AI?**

- **Short Answer:** Spiral is a risk-driven model where each revolution expands the product while evaluating technical and operational risks.
- **Detailed Answer:** If developed under Spiral: Circuit 1 evaluated the risk of AI hallucination via simple prompt prototypes. Circuit 2 resolved source authority risks by adding Google Search grounding. Circuit 3 resolved UI responsiveness risks with the Verification Pulse loader. Circuit 4 resolved data loss and security risks with SQLite persistence and JWT auth.
- **SEPM Concept:** Spiral Model; Anchor-Point Milestones; Risk-Driven Process.

**Q16: What is the Concurrent Development Model, and did you use it?**

- **Short Answer:** Yes; the frontend and backend existed concurrently in different states (e.g., backend in "awaiting testing" while frontend was "under development").
- **Detailed Answer:** Concurrent engineering models activities as existing simultaneously in states such as *inactive*, *under development*, *awaiting changes*, or *done*. In TrustLens, while the backend authentication was *done*, the frontend OAuth UI transitioned from *under development* to *review*, triggered by state events.
- **SEPM Concept:** Concurrent Engineering; Activity States.

**Q17: What are Component-Based Software Engineering (CBSE) practices in TrustLens?**

- **Short Answer:** TrustLens reuses pre-built npm libraries ( `@google/genai` , `lucide-react` , `better-sqlite3` , `jsonwebtoken` ) rather than writing low-level drivers from scratch.
- **Detailed Answer:** CBSE follows 5 steps: researching components, evaluating integration, designing architecture to fit them, integrating, and testing. We integrated established open-source components, compressing development time while maintaining high reliability.
- **SEPM Concept:** Component-Based Development; Software Reuse.

**Q18: What is Aspect-Oriented Software Development (AOSD) and where is it in your project?**

- **Short Answer:** AOSD handles "crosscutting concerns" like authentication, error logging, and security headers that span across multiple modules.
- **Detailed Answer:** In our Express backend, token authentication ( `requireUser` middleware in `backend/src/routes/me.js` ) and CORS handling are crosscutting concerns implemented as middleware functions that intercept all routes rather than being duplicated in each controller.
- **SEPM Concept:** Crosscutting Concerns; Separation of Concerns.

**Q19: Explain the difference between an Incremental Model and an Evolutionary Model.**

- **Short Answer:** Incremental delivers complete, usable functional subsets sequentially; Evolutionary delivers increasingly refined versions of the entire system in circular passes.
- **Detailed Answer:** In an Incremental model, Increment 1 delivers full basic chat; Increment 2 adds file parsing. In an Evolutionary model (like Prototyping or Spiral), early versions may be rough mock-ups across all functions that evolve into complete, robust systems through iterative refinement.
- **SEPM Concept:** Incremental Process vs. Evolutionary Process.

**Q20: What are Ambler's three types of Process Patterns?**

- **Short Answer:** (1) Stage Pattern (framework activity), (2) Task Pattern (action or task), and (3) Phase Pattern (sequence of framework activities like Prototyping).
- **Detailed Answer:** Ambler defined process patterns to describe recurring development challenges. A Task pattern in TrustLens was "Grounding Source Extraction", which established a reusable method for parsing raw Gemini candidates into verified citation arrays.
- **SEPM Concept:** Ambler's Process Patterns; Pattern Template.

**Q21: What is SCAMPI and CMMI in software process assessment?**

- **Short Answer:** SCAMPI is a five-phase assessment method used to evaluate an organization's process maturity against SEI CMMI levels.
- **Detailed Answer:** SCAMPI (Standard CMMI Assessment Method for Process Improvement) uses *initiating, diagnosing, establishing, acting, learning* to determine process maturity from Initial (Level 1) to Optimizing (Level 5).
- **SEPM Concept:** Process Assessment & Improvement (Module 1 Notes §2.2).

**Q22: What are the 12 Agile Principles, and which one was most vital to TrustLens?**

- **Short Answer:** Principle 7: "*Working software is the primary measure of progress.*"
- **Detailed Answer:** Rather than producing lengthy specification documents that describe hypothetical verification algorithms, our primary milestone was always a functional, working build capable of taking real user questions and returning verified claim cards.
- **SEPM Concept:** Agile Manifesto; Principles of Agility.

**Q23: Explain DSDM and the 80/20 Pareto Principle in your project.**

- **Short Answer:** DSDM states 80% of an application's value can be delivered in 20% of the time required for a 100% complete solution.
- **Detailed Answer:** In TrustLens AI, the core 80% business value was accurate claim extraction, source grounding, and clear confidence scores. The remaining 20% (social sharing, transcript printing, advanced export) was deferred to later iterations.
- **SEPM Concept:** Dynamic Systems Development Method (DSDM); Timeboxing.

**Q24: What is Lean Software Development (LSD) and how do you "eliminate waste"?**

- **Short Answer:** Eliminating waste means building no extraneous features, avoiding unnecessary documentation, and stopping defect leakage early.
- **Detailed Answer:** In TrustLens AI, we avoided "gold plating" (adding unnecessary decorative widgets that provide no user value) and used automated tests so defects were caught immediately instead of accumulating costly rework.
- **SEPM Concept:** Lean Software Development; Waste Elimination.

---

## Category C: Project Management & Control (Q25–Q36)

### Q25: What is the difference between SDLC and PMLC?

- **Short Answer:** SDLC covers engineering activities (requirements to deployment); PMLC oversees the entire endeavor (from feasibility and chartering to closure).
- **Detailed Answer:** As shown in Notes Figure 1.7, the Project Management Life Cycle begins before the SDLC (with concept development, feasibility study, and project chartering) and continues during and after SDLC execution (monitoring cost/time, managing contracts, closing deliverables, and post-implementation reviews).
- **SEPM Concept:** Project Management Life Cycle vs. SDLC (Notes Page 20).

### Q26: What are SMART objectives in Software Project Management?

- **Short Answer:** Objectives must be Specific, Measurable, Achievable, Relevant, and Time-constrained.
- **Detailed Answer:** In TrustLens AI:
  - *Specific:* Build an evidence-grounded verification chat application.
  - *Measurable:* Achieve sub-3-second API response time and 100% test pass rate across all 89 test cases.
  - *Achievable:* Use established Gemini APIs and Node.js rather than training custom foundational LLMs.
  - *Relevant:* Directly addresses the critical problem of student/researcher misinformation.
  - *Time-constrained:* Complete and test all 4 increments within the 12-week semester.
- **SEPM Concept:** Setting Objectives (Notes §1.9).

### Q27: What is the Standish Group CHAOS Report, and why does it matter?

- **Short Answer:** A benchmark study finding only ~30% of software projects succeed; the majority fail or run over budget due to poor project management.
- **Detailed Answer:** The Standish Group showed that 82% of projects ran late and 43% exceeded budgets. This proves that technical coding skill alone cannot guarantee success—diligent scope management, risk mitigation, and tracking are mandatory.
- **SEPM Concept:** Importance of SPM (Notes §1.2).

### Q28: How did you identify and mitigate technical risks in TrustLens AI?

- **Short Answer:** We maintained a proactive Risk Management Plan covering API rate limits, schema changes, and response latency.
- **Detailed Answer:**
  - *Risk 1 (Gemini Rate Limits / Outages):* Mitigated by implementing client-side retry logic and clear HTTP 503 error status.
  - *Risk 2 (Non-Deterministic JSON responses):* Mitigated by enforcing strict `responseSchema` parameters in the Gemini API call.
  - *Risk 3 (Data Loss on Session End):* Mitigated by implementing guest account migration to SQLite.
- **SEPM Concept:** Risk Management & Containment; Umbrella Activity.

**Q29: What is Scope Creep and how did you prevent it in TrustLens AI?**

- **Short Answer:** Scope creep is the uncontrolled expansion of product requirements without adjustments to time, cost, or resources.
- **Detailed Answer:** During development, ideas emerged to add multi-user live collaborative channels and PDF annotation highlights. Because our project schedule was fixed, we implemented strict **Change Management**: recording new feature requests in a "Future Enhancements" log while keeping the active sprint focused strictly on baseline verification and authentication.
- **SEPM Concept:** Scope Management; Baseline Control.

**Q30: What is the Business Case for TrustLens AI?**

- **Short Answer:** To save users time and prevent financial/academic loss caused by relying on hallucinated AI information.
- **Detailed Answer:** If a student relies on a generic chatbot and misses an exam deadline or pays an unapproved fee, the cost is catastrophic. TrustLens AI provides verifiable citations from official sources, delivering tangible risk-reduction value that justifies its operational infrastructure costs.
- **SEPM Concept:** The Business Case (Notes §1.10).

**Q31: What are RFQ, RFP, and RFI in Project Bidding?**

- **Short Answer:** RFI assesses vendor competencies; RFP requests proposed solutions when the solution is not fully known; RFQ requests price quotations for well-understood specifications.
- **Detailed Answer:** If TrustLens were outsourced: We would publish an RFP to evaluate how different AI vendors handle search grounding, followed by an RFQ to select the most cost-effective cloud provider for hosting.
- **SEPM Concept:** Project Bidding Techniques (Notes §1.13).

**Q32: Explain Extreme Project Management (XPM).**

- **Short Answer:** A modern project management approach that plans short, incremental deliveries with evolving requirements, emphasizing human collaboration over rigid long-term plans.
- **Detailed Answer:** Instead of drafting a rigid 6-month Gantt chart upfront, XPM embraces high-change environments by planning short iterations and managing stakeholders actively throughout the project lifecycle.
- **SEPM Concept:** Traditional vs. Modern Management Practices (Notes §1.14).

**Q33: How does Contract Management apply to TrustLens AI?**

- **Short Answer:** Through Terms of Service, API license agreements with Google Cloud, and third-party data compliance contracts.
- **Detailed Answer:** TrustLens relies on the Google Cloud API contract. Contract management involves monitoring quota obligations, ensuring GDPR/DPDP data compliance, and avoiding unexpected billing spikes.
- **SEPM Concept:** Contract Management Lifecycle (Notes §1.4).

**Q34: What is MTBF, and how is it used in software?**

- **Short Answer:** Mean Time Between Failures—a metric of software reliability measured over operational execution time.

- **Detailed Answer:** In TrustLens, MTBF measures how long the server runs and processes user queries before encountering an unhandled exception or crash. We achieved high MTBF by wrapping asynchronous routes in centralized error-handling middleware.
- **SEPM Concept:** Measures of Effectiveness (Notes §1.9.2).

**Q35: What is Configuration Management (SCM) in TrustLens AI?**

- **Short Answer:** Controlling, versioning, and tracking changes to source code, database schemas, and configuration files.
- **Detailed Answer:** We used Git for commit tracking, tagged release baselines, and maintained isolated configuration files ( `.env` vs `.env.example` ) so that production credentials were never checked into version control.
- **SEPM Concept:** Software Configuration Management (SCM); Baselines.

**Q36: What happens during Project Closure?**

- **Short Answer:** Releasing deliverables, archiving documentation, terminating temporary resources, and conducting a post-implementation review.
- **Detailed Answer:** In TrustLens AI, project closure included creating the final deliverable UI PDF ( `TrustLens-Complete-UI.pdf` ), verifying all test suites pass, archiving the database migration scripts, and documenting lessons learned.
- **SEPM Concept:** Project Closure (Notes §1.13.6).

---

## Category C: Verification, Validation & Testing (Q37–Q50)

**Q37: What is the exact difference between Verification and Validation in TrustLens AI?**

- **Short Answer:** *Verification:* "Are we building the product right?" (code reviews, unit tests, schema conformance). *Validation:* "Are we building the right product?" (does it meet real user fact-checking needs?).
- **Detailed Answer:**
  - **Verification:** Internal engineering checks. Did our functions execute without errors? Does `calibrateTrustAnswer` adhere to its mathematical specification? Do unit tests pass?
  - **Validation:** User-centric testing. Does the answer actually satisfy the student's need to verify their eligibility? Is the evidence cited actually authoritative in the real world?
- **SEPM Concept:** V&V (ISO 12207 / Notes §1.5.3.5).

**Q38: What testing levels did you implement in TrustLens AI?**

- **Short Answer:** Unit testing, Integration testing, System testing, and Acceptance testing.
- **Detailed Answer:**
  1. *Unit Testing:* Math formulas in `confidence.js`.
  2. *Integration Testing:* Database user insertion coupled with JWT authentication routes in `auth.js`.
  3. *System Testing:* End-to-end flow from message submission through Gemini grounding to browser rendering.
  4. *Acceptance Testing:* Verifying that student users can clearly read claim cards and switch themes without contrast failure.

- **SEPM Concept:** Testing Levels (Notes Page 12 & 25).

**Q39: What is the Pareto Principle in software testing?**

- **Short Answer:** 80% of all software defects are found in 20% of the program components.
- **Detailed Answer:** In TrustLens, 80% of edge-case bugs occurred in two critical modules: the asynchronous Gemini streaming handler and the complex CSS contrast classes in the chat interface. Isolating and thoroughly testing these two modules eliminated the vast majority of application errors.
- **SEPM Concept:** Davis Principle 3 (Notes Page 26).

**Q40: What is a Formal Technical Review (FTR), and why is it called a "Quality Filter"?**

- **Short Answer:** A structured review of design models, code, or documentation by peers to detect and remove defects before testing.
- **Detailed Answer:** Industry data shows that 60–80% of defects can be caught through formal reviews before a single test case is executed. Reviewing our prompt instructions and database schemas early prevented major architectural rework during coding.
- **SEPM Concept:** FTR as Quality Filter (Notes Page 7).

**Q41: Can software testing prove the absence of defects?**

- **Short Answer:** No. As Glen Myers and Edsger Dijkstra stated, testing can only show the presence of defects, never their absence.
- **Detailed Answer:** Davis Principle 5 states that exhaustive testing is impossible due to infinite permutations of input. We achieved high reliability through equivalence partitioning and boundary value testing across representative user queries.
- **SEPM Concept:** Glen Myers' Testing Rules; Davis Principle 5.

**Q42: What is Regression Testing, and when is it conducted?**

- **Short Answer:** Re-executing existing tests after code modifications to ensure changes have not introduced new defects.
- **Detailed Answer:** Whenever we updated the CSS theme tokens or modified user authentication, we re-ran our full automated test suite ( `scripts/test-google-auth.js` and API audits) to verify zero regressions.
- **SEPM Concept:** Regression Testing; SCM.

**Q43: How do you handle non-functional requirements (NFRs) in TrustLens AI?**

- **Short Answer:** We engineered for Performance (<3s response), Security (scrypt/JWT), Accessibility (WCAG AAA), and Usability.
- **Detailed Answer:** NFRs dictate system quality rather than specific behaviors. We measured performance through network benchmarks, enforced security through input sanitation and token hashing, and validated accessibility through automated axe audits and contrast testing.
- **SEPM Concept:** Non-Functional Requirements (NFRs); Software Quality.

**Q44: What is Test-Driven Development (TDD)?**

- **Short Answer:** A development technique where unit tests are written before the functional code is implemented.

- **Detailed Answer:** In TDD, you write a failing test, write the minimal code to pass the test, and then refactor. We used TDD when developing the confidence calibration algorithm to verify that corner cases (e.g., zero sources, conflicting sources) produced exact expected scores.
- **SEPM Concept:** XP Coding Practice; TDD.

**Q45: How do you measure the quality of a Web Application according to your syllabus?**

- **Short Answer:** Network intensiveness, concurrency, performance, availability, data-driven content, and aesthetics.
- **Detailed Answer:** Module 1 (§1.3) defines Web App attributes: responsiveness, look-and-feel aesthetics, and continuous evolution. In TrustLens, we prioritized high performance and intuitive visual hierarchy to ensure an accessible, professional user experience.
- **SEPM Concept:** Unique Nature of Web Apps (Notes §1.3).

**Q46: What is Cleanroom Software Engineering?**

- **Short Answer:** A formal methods approach that uses rigorous mathematical specification and statistical verification to build defect-free software without conventional unit testing.
- **Detailed Answer:** While cleanroom development is common in life-critical medical or avionics software, it is too expensive and slow for commercial web applications like TrustLens AI.
- **SEPM Concept:** Formal Methods Model (Notes §2.7).

**Q47: What is Pair Programming, and what are its tangible benefits?**

- **Short Answer:** Two programmers working at one workstation—one "Driver" writing code, one "Navigator" reviewing in real time.
- **Detailed Answer:** XP pair programming improves code quality, reduces defect rates by up to 50%, facilitates continuous design reviews, and spreads domain knowledge across team members.
- **SEPM Concept:** Extreme Programming Practices (Notes Page 8).

**Q48: How did you implement Change Management when requirements shifted?**

- **Short Answer:** By evaluating each change request against project scope, impact on schedule, and architectural risk before implementing.
- **Detailed Answer:** When the requirement changed to add Google OAuth, we performed an impact assessment: schema changes, security implications, and frontend button additions. We implemented the change in an isolated branch and verified regression safety.
- **SEPM Concept:** Change Management / Configuration Management (Notes §1.14).

**Q49: If you had another month, what would be your next engineering increment?**

- **Short Answer:** Integrating automated multi-LLM consensus verification (comparing Gemini with Claude/OpenAI) and local vector embeddings for offline PDF search.
- **Detailed Answer:** In Increment 5, we would implement Retrieval-Augmented Generation (RAG) using local vector embeddings (ChromaDB/Faiss), enabling users to query 500-page institutional rulebooks offline with exact page-level source citations.
- **SEPM Concept:** Continuous Evolution; Evolutionary Process Model.

**Q50: How do you conclude your viva with maximum academic impact?**

- **Winning Closing Statement:**

> *"Madam, TrustLens AI is not merely a conversational wrapper around an LLM. It is an engineered software product built to solve the fundamental problem of AI unreliability. By applying the principles of Software Engineering and Project Management (24CS51)—from an Agile incremental development model and strict component modularity to deterministic confidence calibration and automated regression testing—we transformed an unpredictable AI service into a disciplined, auditable, and user-centered verification engine."*

---

## 9. EXAMINER TRAPS, RED LINES & HOW TO ANSWER HONESTLY

| Examiner Trap / Question | Dangerous / Wrong Answer | Academic & Honest Winning Answer |
|---|---|---|
| *"Who was your Scrum Master?"* | *"I was the Scrum Master and my friend was the Product Owner."* (Easily exposed as fake). | *"Madam, as an academic project team, we did not run formal corporate Scrum ceremonies with a full-time Scrum Master. Instead, we adopted **Agile principles from XP and FDD**: 2-week feature-driven iterations, spike solutions for AI risk, and continuous unit testing."* |
| *"Does your AI guarantee 100% truth?"* | *"Yes, our AI never makes mistakes because it uses Google Search."* | *"No, madam. Generative AI is inherently probabilistic. What TrustLens AI guarantees is **transparency**: we extract the claims, present the exact supporting sources, calibrate confidence mathematically, and explicitly highlight what remains unverified."* |
| *"Why didn't you write a 200-page SRS document first?"* | *"We didn't have time."* | *"Under Agile principles, working software is the primary measure of progress. Because the external AI API and UI requirements were evolving, drafting a rigid upfront document would have violated Hooker's KISS principle and created obsolete documentation. We maintained living API specifications and living test suites instead."* |
| *"Why is your database in SQLite instead of Cloud SQL?"* | *"It was easier."* | *"It was an architectural decision based on Hooker's KISS principle and Davis's design principles. SQLite provides zero-latency ACID transactions, requires no network socket overhead, and is embedded directly within our Node process, making it ideal for our deployment scope."* |